HOCHSCHULE
MITTWEIDA
University of Applied Sciences

BACHELOR THESIS

Mr.
Georgii Burdin

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification

Mittweida, April 2026

Faculty of **Applied Computer Sciences and Biosciences**

BACHELOR THESIS

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification

Author:
Georgii Burdin

Course of Study:
B.Sc. Applied Mathematics

Seminar Group:
MA22w1-B

First Examiner:
Prof <Name> <First Examiner>

Second Examiner:
<Second Examiner>

Submission:
Mittweida, 01.04.2026

Defense/Evaluation:
Mittweida, 2026

Bibliographic Description

Burdin, Georgii:
Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification. – 2026. – 13 S.
Mittweida, Hochschule Mittweida – University of Applied Sciences, Faculty of Applied Computer Sciences and Biosciences, Bachelor Thesis, 2026.

Referat

Large language model APIs are silently updated by their providers, causing *model drift*—measurable changes in output quality over time without prior notice to downstream users. Chen et al. (2023) demonstrated significant performance degradation in ChatGPT across several tasks within months. This thesis builds a lightweight, reproducible benchmarking framework that periodically evaluates multiple LLM APIs (OpenAI, Gemini, Claude) on a fixed campaign-relevance classification dataset, detects drift via disagreement-based sampling, and quantifies accuracy–cost trade-offs across providers.

Contents

List of Figures and Tables

List of Tables

1 Introduction

Most commercially available large language models are deployed as continuously updated API services. Consequently, their behaviour and task-specific performance may change over time even when the public API interface remains unchanged. Empirical evidence indicates that such updates may lead either to improvements or to degradation on individual tasks. This phenomenon, commonly described as *model drift* or *update regression*, necessitates continuous benchmarking and monitoring in production settings.

Chen et al. [1] reported substantial changes in the behaviour of GPT-3.5 and GPT-4 across several tasks between March and June 2023. In particular, the authors observed a pronounced decline in GPT-4 performance on prime-number classification during that period. For production systems that rely on LLM APIs as classification components, such silent changes constitute an operational risk, since downstream business metrics may become unstable without explicit notification from the model provider.

Against this background, the central problem is the absence of a lightweight and reproducible procedure that allows API consumers to monitor whether the model behind a deployed endpoint continues to satisfy task-specific quality requirements over time.

The following research questions are addressed:

1. To what extent does the classification accuracy of selected LLM APIs vary over time when evaluated on a fixed campaign-relevance dataset?
2. Can disagreement-based sampling reduce evaluation costs while preserving sensitivity to performance changes?
3. How do the considered models compare with respect to the trade-off between predictive quality and evaluation cost?

The main contribution is a reproducible benchmarking pipeline that integrates a labelled multimodal dataset in Langfuse, dispatches evaluation batches to multiple model providers, records accuracy and cost metrics over time, and enables the detection of statistically relevant performance changes.

2 Theoretical Background

2.1 Model Drift and Concept Drift

In the literature on learning systems, *concept drift* denotes a change in the conditional distribution $P(Y \mid X)$ over time. In practical terms, this means that the relationship between inputs and target labels is no longer stable. The primary object of interest here is not concept drift in the incoming data stream, but *model drift*: changes in model behaviour caused by updates to the underlying model while the evaluation data remain fixed. This distinction allows observed performance changes to be attributed to the model side rather than to changes in the task distribution.

Chen et al. [1] provide empirical evidence that the behaviour of GPT-3.5 and GPT-4 changed substantially over time on tasks such as mathematical reasoning, code generation and sensitive-question handling. Their findings motivate the use of fixed reference datasets when the objective is to measure temporal changes in the performance of continuously updated LLM APIs.

2.2 LLM Performance Degradation

Performance degradation in LLM APIs may arise for several reasons, including changes in the underlying model weights, quantisation, distillation, renewed post-training alignment, or provider-side system adjustments. Regardless of the technical cause, such interventions may alter the effective input–output mapping of the deployed system. In contrast to many classical machine learning deployments, users of commercial LLM APIs typically do not have access to a fixed checkpoint and therefore cannot directly control model versioning at the parameter level. The remaining practical option is to monitor outputs and evaluate whether task-specific performance remains stable over time.

2.3 Drift Detection Methods

Common approaches to drift detection include statistical process control procedures, such as CUSUM and ADWIN, distributional monitoring based on measures such as the population stability index, and supervised comparison methods based on labelled outcomes. Since the present study evaluates repeated model predictions against fixed ground-truth labels, the most relevant tools are interval-based monitoring of accuracy estimates and paired significance tests for binary outcomes. Accordingly, the empirical analysis relies primarily on confidence intervals for accuracy and on McNemar's test for paired comparisons between models or between repeated evaluations of the same model.

2.4 LLM-as-a-Judge and Evaluation Frameworks

The experimental framework combines batch inference APIs provided by OpenAI, Google Gemini and Anthropic with Langfuse as the central platform for dataset management, trace collection and score logging. This setup enables repeated evaluation in a reproducible manner and supports the joint analysis of predictive performance and token-based costs.

3 Data and Task Definition

3.1 Business Task

The application scenario arises in the context of Instagram-based advertising campaigns for a brand. During such campaigns, users publish posts containing images and captions, some of which are directly related to the promoted brand, products or campaign creatives, while others are only weakly related or entirely unrelated. From a business perspective, the objective is to determine whether a user-generated post should be counted as part of the effective campaign reach.

This problem is formalised as a binary classification task. Given a post consisting of an image and an associated caption, the classifier predicts the label `campaign_relevant` $\in$ $\{$`true`, `false`$\}$. Reliable classification is important because the resulting labels enter downstream analyses such as reach estimation, noise filtering and campaign reporting. Since the task requires joint interpretation of visual and textual content, LLM APIs constitute a suitable modelling choice. At the same time, their use complicates evaluation, because silent provider-side model updates may alter classification quality and thereby affect business conclusions.

3.2 Dataset Collection

The dataset is derived from real Instagram posts associated with a brand-related advertising context. Each example consists of:

- an image referenced via a content-delivery network (CDN) URL of the form `https://d3lbkpzyekd5x`
- the corresponding post caption,
- a human-provided binary label `campaign_relevant` $\in$ $\{$`true`, `false`$\}$.

Human annotators were instructed to assign the positive label when a post clearly referred to the campaign's brand, product or official visual identity, and the negative label otherwise. Ambiguous cases were resolved either through majority voting or through escalation to an expert annotator. This annotation procedure was intended to produce a stable reference set for repeated comparative evaluation.

For integration into the benchmarking pipeline, the dataset is stored in Langfuse. Each item contains a unique `custom_id`, a `body` field encoding the multimodal prompt input, and the ground-truth label `campaign_relevant`. After collection and cleaning, the dataset is held fixed throughout the experimental period: no new items are introduced and no labels are modified. This design decision is essential for the interpretation of the results, because it allows observed changes in accuracy to be attributed to model drift rather than to changes in the data-generating process.

3.3 Dataset Size and Distribution

The final dataset contains $N = 1{,}200$ labelled examples. Of these, 682 are labelled as campaign relevant and 518 as not relevant, corresponding to class proportions of approximately $56.8\,\%$ and $43.2\,\%$, respectively. This moderate class imbalance is plausible in the present application, where a substantial share of user-generated content is related to the campaign, while a non-negligible portion remains off-topic.

This collection of $1{,}200$ labelled posts is treated as the fixed reference population against which all evaluations are defined. Consequently, temporal changes in predictive performance refer to changes with respect to the same underlying set of examples.

3.4 Evaluation Subset Selection

Evaluating every model on all $1{,}200$ items in every run would be too costly. Instead, the benchmark uses a fixed evaluation subset of size $n = 300$, chosen once to concentrate budget on informative items.

All $N = 1{,}200$ examples are first evaluated once by all models. For each item i, the binary predictions $\hat{y}_{i1}, \ldots, \hat{y}_{iK}$ from K models are mapped to $\{0, 1\}$ and summarised by the disagreement score

$$s_i = \sigma(\hat{y}_i) \cdot H(p_i),$$

where $\sigma(\hat{y}_i)$ denotes the standard deviation of the K binary predictions and

$$H(p_i) = -\,p_i \log p_i - (1 - p_i) \log(1 - p_i)$$

is the binary entropy associated with the proportion p_i of positive predictions. Items are partitioned by the percentile of s_i into high-disagreement (above the 80th percentile), medium-disagreement (between the 50th and 80th percentiles) and low-disagreement (below the 50th percentile) groups.

From these three strata, a one-time sample of $n = 300$ is drawn using fixed allocation weights of approximately $67\,\%$ (high disagreement), $23\,\%$ (medium disagreement) and $10\,\%$ (low disagreement). This subset is stored as a dedicated evaluation dataset and remains unchanged for all subsequent benchmark runs.

3.5 Statistical Justification of the Subset Size

The choice $n = 300$ is derived from two complementary statistical requirements imposed on the experimental design.

3.5.1 Precision of Single-Model Accuracy Estimates

First, the evaluation subset must yield an estimate of single-model accuracy with sufficiently high precision. Let $\hat{p}$ denote the observed accuracy on a subset of size n. Under the standard normal approximation to the binomial distribution, a two-sided $(1-\alpha)$ confidence interval for p is given by

$$\hat{p} \pm z_{1-\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}},$$

where $z_{1-\alpha/2}$ denotes the corresponding standard normal quantile. Requiring a target half-width h implies

$$h \approx z_{1-\alpha/2}\sqrt{\frac{p(1-p)}{n}},$$

which leads to the sample-size condition

$$n \geq \frac{z_{1-\alpha/2}^2\, p(1-p)}{h^2}.$$

To obtain a conservative bound, $p = 0.5$ is used, since this maximises the variance term $p(1-p)$. With $\alpha = 0.05$ and target half-width $h = 0.06$, the resulting lower bound is approximately $n \geq 267$. The selected subset size therefore satisfies the required precision criterion.

3.5.2 Power to Detect Accuracy Differences Between Models

Second, the subset must provide sufficient statistical power to detect practically relevant performance differences between two models evaluated on the same examples. Let δ denote the minimum accuracy difference of interest and let q denote the probability that the two models differ in correctness on a given item, that is, that one model is correct while the other is not. For paired binary outcomes, a standard approximation yields the sample-size requirement

$$n \geq \frac{(z_{1-\alpha/2} + z_{1-\beta})^2\, q}{\delta^2}.$$

In the present study, $\alpha = 0.05$, the desired power is set to $1-\beta = 0.8$, the minimum relevant difference is $\delta = 0.05$, and the empirical discordance rate is approximately $q = 0.2$. Substituting these values gives a lower bound of roughly $n \geq 251$. Hence, the chosen subset size also satisfies the power requirement for paired comparisons.

Taken together, these calculations show that a subset size of $n = 300$ provides a reasonable compromise between statistical precision, sensitivity to relevant performance changes, and the practical cost constraints associated with repeated API-based evaluation. In the present study, this subset size characterises the fixed evaluation set that is used across all benchmark runs.

4 Methodology

4.1 Disagreement-Based Sampling

The disagreement-based scheme described above is used once to construct the fixed evaluation subset. A preliminary full run over all $1{,}200$ examples assigns each item a disagreement score s_i and a stratum (high, medium or low disagreement). The subset of size $n = 300$ is then drawn from these strata using the fixed $67/23/10$ allocation and materialised as a separate dataset.

This concentrates the subset on items where model decisions are unstable, while still retaining coverage of the broader reference population.

4.1.1 Methodological Note: Benchmarking and Sampling Design

Using a reduced subset is compatible with a fixed benchmark when one distinguishes between a fixed *reference population* and a fixed *evaluation design*. The population consists of all $1{,}200$ labelled posts; the subset of size $n = 300$ is derived once from this population by the disagreement-based rule and then held fixed.

Repeated evaluations therefore estimate accuracy under the evaluation distribution induced by this design. The confidence-interval and power calculations in Section 3.5 remain applicable, since they describe the properties of samples of size $n = 300$ drawn under the same design; the realised subset is one such sample that is reused across runs.

Several related sample-selection strategies have been discussed in the literature. Uniform random sampling aims at unbiased coverage of the reference population but may allocate excessive budget to easy examples that contribute little to change detection. Uncertainty sampling selects items near the decision threshold of a single model. Query-by-committee and related disagreement-based procedures select observations for which several models disagree, thereby concentrating attention on regions of epistemic uncertainty or instability [2]. Dynamic benchmark approaches, such as Dynabench, further argue that informative evaluation may require targeted and evolving test sets rather than a purely static benchmark [3].

The adopted method is most closely related to query-by-committee. This choice is motivated by the structure of the task and by the empirical label distribution. Since the objective is not merely to estimate average accuracy, but to detect changes in model behaviour under a constrained evaluation budget, it is advantageous to over-represent observations near the empirical disagreement boundary. At the same time, the inclusion of medium- and low-disagreement strata prevents the evaluation from collapsing onto a narrow subset of highly contentious examples. The chosen $67/23/10$ allocation should therefore be interpreted as a design decision that balances detection sensitivity with population coverage, rather than as a claim of statistical optimality.

4.2 Batch Evaluation Pipeline

For each provider, the evaluation workflow consists of constructing a JSONL batch request, submitting it to the corresponding batch API, polling until completion, parsing the returned predictions, computing accuracy with respect to the ground-truth labels, and storing the resulting traces and scores in Langfuse.

4.3 Drift Detection

Drift detection is based on changes in accuracy across successive runs on the fixed subset of $n = 300$ examples. The monitoring dashboard raises an operational alert when the absolute accuracy difference between two runs exceeds one percentage point, that is, about three items out of 300.

Statistical relevance is assessed separately: a change is treated as statistically relevant when the confidence interval of the accuracy difference excludes zero or when McNemar's test on the paired contingency table yields a p-value below the chosen significance level. The sample-size calculations in Section 3.5 show that $n = 300$ offers sufficient precision and power to detect practically meaningful accuracy differences while keeping evaluation costs manageable.

5 Experimental Setup

The experimental comparison covers a mixture of frontier and lightweight models from the OpenAI, Gemini and Claude families. Concretely, the evaluated set comprises the OpenAI models `gpt-5.4-2026-03-05`, `gpt-5.2`, `gpt-5.1`, `gpt-5`, `gpt-5-mini`, `gpt-5-nano`, `gpt-4.1`, `gpt-4.1-mini`, and `gpt-4.1-nano`; the Gemini models `gemini-2.5-flash`, `gemini-2.5-flash-previe` `gemini-2.0-flash`, `gemini-3-flash-preview`, and `gemini-3-pro-preview`; and, where available, recent Claude sonnet and haiku variants. All providers are evaluated using an identical system prompt and a common structured-output schema. The benchmark is repeated periodically over N runs, and evaluation cost is recorded on the basis of input and output token counts together with batch-API prices as listed in the configuration file.

6 Results

The results chapter presents temporal accuracy curves for each model together with $95\,\%$ Wilson confidence intervals, pairwise disagreement heatmaps, and the observed accuracy–cost trade-off across providers. A baseline run on $n = 300$ examples serves as the reference point for the subsequent time series; its aggregate accuracy and cost figures per model are summarised in Table 6.1. Detected drift events are reported together with their estimated effect sizes and corresponding significance measures. In addition, the chapter refers to the Streamlit dashboard developed for continuous monitoring.

6.1 Aggregate Accuracy and Cost per Model

Table 6.1 reports accuracy and batch-evaluation cost per model for the baseline run of the benchmark. The evaluation subset contains $n = 300$ examples drawn according to the disagreement-based sampling design described in Chapter *Methodology*. Costs correspond to the realised batch charges per run and are rounded to two decimal places.

Within the OpenAI family, `gpt-5` attains the highest observed accuracy (0.9758), closely followed by `gpt-5.2` and `gpt-5-mini`. From a cost-adjusted perspective, `gpt-5-mini` and `gpt-5-nano` dominate the older `gpt-4.1` variants, offering substantially better accuracy at comparable or lower batch costs, while `gpt-4o` and `gpt-4o-mini` are clearly dominated both in terms of accuracy and cost.

Among the Gemini models, `gemini-2.5-flash` and its preview snapshot outperform `gemini-2.0-flash` by a wide margin while remaining inexpensive in absolute terms. The `gemini-3-`$*$ previews, by contrast, are less accurate and substantially more costly, and therefore do not lie on the empirical accuracy–cost Pareto frontier for this task.

Taken together, the baseline run suggests that, for campaign-relevance classification under a constrained evaluation budget, the most attractive operating points are achieved by `gpt-5-mini`, `gpt-5-nano`, `gemini-2.5-flash` and `gemini-2.5-flash-preview-09-2025`. Frontier models such as `gpt-5` and `gpt-5.2` deliver the highest absolute accuracy, but the incremental gains over the best lightweight models must be weighed against a several-fold increase in batch cost.

Table 6.1: Accuracy and batch-evaluation cost per model on the campaign-relevance classification subset ($n = 300$). Costs are rounded to two decimal places.

Model	Accuracy	Cost per run (USD)
`gemini-2.0-flash`	0.6297	1.69
`gemini-2.5-flash`	0.9408	0.73
`gemini-2.5-flash-preview-09-2025`	0.9331	0.25
`gemini-2.5-pro`	0.9216	2.86
`gemini-3-flash-preview`	0.6564	7.97
`gemini-3-pro-preview`	0.8190	17.42
`gpt-4.1`	0.9440	9.86
`gpt-4.1-mini`	0.8471	2.44
`gpt-4.1-nano`	0.7648	0.67
`gpt-4o`	0.8672	12.02
`gpt-4o-mini`	0.8329	16.32
`gpt-5`	0.9758	12.70
`gpt-5-mini`	0.9682	1.73
`gpt-5-nano`	0.9464	0.46
`gpt-5.1`	0.9365	5.81
`gpt-5.2`	0.9670	8.29

7 Discussion

The discussion interprets the observed drift patterns, or the absence thereof, in relation to the findings reported by Chen et al. [1]. Particular attention is given to the question of whether disagreement-based sampling reduced evaluation costs without materially weakening the ability to detect performance changes. The limitations of the study include the restriction to a single binary classification task, the finite observation window, and the absence of provider-side information about internal model updates.

8 Conclusion and Outlook

The concluding chapter summarises the answers to the research questions and formulates practical recommendations for the use of LLM APIs in monitoring scenarios. In particular, it discusses the importance of version pinning where available and of periodic benchmark evaluations where such control is absent. Possible directions for future work include extensions to multi-class settings, automated alerting mechanisms, and longer-term monitoring over extended time horizons.

Appendix A: Implementation Details

This appendix documents the main implementation components of the benchmarking pipeline, including the scripts `sample.py`, `run_eval.py`, `batch_openai.py`, `batch_gemini.py`, `batch_claude.py`, `poll_and_ingest.py` and `report.py`. It also summarises the Streamlit monitoring application in `streamlit_app/app.py` and the relevant configuration files, in particular `config/models.yaml`.

In the deployed setup, the fixed evaluation subset of $n = 300$ items is materialised as a dedicated dataset in Langfuse and, where convenient, as a local CSV file. The script `run_eval.py` always operates on this subset, either by loading the corresponding Langfuse dataset or by reading the CSV, so that all reported runs are directly comparable.

Appendix B: Additional Plots and Tables

This appendix contains extended accuracy tables, additional visualisations, alternative drift-detection thresholds and detailed token-cost breakdowns that complement the results discussed in the main text.

Bibliography

[1] L. Chen, M. Zaharia, and J. Zou, “How is ChatGPT’s behavior changing over time?”, *arXiv preprint arXiv:2307.09009*, 2023.

[2] B. Settles, “Active learning literature survey”, University of Wisconsin–Madison, Tech. Rep., 2009.

[3] D. Kiela, M. Biasiucci, A. Williams, D. de Las Casas, et al., “Dynabench: Rethinking benchmarking in NLP”, in *Proceedings of NAACL*, 2021.

Statutory Declaration in Lieu of an Oath

I – Georgii Burdin – do hereby declare in lieu of an oath that I have composed the presented work independently on my own and without any other resources than the ones given.

All thoughts taken directly or indirectly from external sources are correctly acknowledged.

This work has neither been previously submitted to another authority nor has it been published yet.

Mittweida, 10. March 2026	
Location, Date	Georgii Burdin